

Kesh Pratap Singh

Capstone Assignment-1 : Report

1. Created new storage account

The screenshot shows the Microsoft Azure portal interface. The top navigation bar includes the Microsoft Azure logo, a search bar, and the user profile 'tredence10@michaelklo...'. The main content area displays the 'Overview' page for the storage account 'keshs2531'. The left sidebar contains a navigation menu with options like 'Overview', 'Activity log', 'Tags', 'Diagnose and solve problems', 'Access Control (IAM)', 'Data migration', 'Events', 'Storage browser', 'Partner solutions', 'Resource visualizer', 'Data storage', 'Security + networking', 'Networking', 'Access keys', and 'Shared access signature'. The main content area shows the 'Essentials' section with a table of account details:

Property	Value
Resource group	LabsKraft2027
Location	southindia
Subscription	LabsKraft
Subscription ID	d008c7cc-9795-4292-bf79-74ae52cda63d
Disk state	Available
Tags	(none)
Performance	Standard
Replication	Locally-redundant storage (LRS)
Account kind	StorageV2 (general purpose v2)
Provisioning state	Succeeded
Created	25/8/2025, 10:27:47 am

Below the Essentials section, there are tabs for 'Properties', 'Monitoring', 'Capabilities (5)', 'Recommendations (0)', 'Tutorials', and 'Tools + SDKs'. The 'Properties' tab is active, showing 'Data Lake Storage' and 'Security' sections.

2. Created a new container and upload blob in raw container

The screenshot shows the Microsoft Azure portal interface for a container named 'raw'. The top navigation bar includes the Microsoft Azure logo, a search bar, and the user profile 'tredence10@michaelklo...'. The main content area displays the 'Overview' page for the container 'raw'. The left sidebar contains a navigation menu with options like 'Overview', 'Diagnose and solve problems', 'Access Control (IAM)', 'Settings', 'Shared access tokens', 'Manage ACL', 'Access policy', 'Properties', and 'Metadata'. The main content area shows the 'raw' container with a table of blobs:

Name	Last modified	Access tier	Blob type	Size	Lease state
retail_iot_data.csv	25/8/2025, 10:33:03 am	Hot (Inferred)	Block blob	130.76 KiB	Available

A notification banner at the top right of the container view states: 'Successfully uploaded blob(s). Successfully uploaded 1 blob(s).' The 'Authentication method' is set to 'Access key'.

3. Created Access Connector

Home > AccessConnectorCreate-20250825103356 | Overview > LabsKraft2027 >

keshconnector

Access Connector for Azure Databricks

Search

RefreshDelete

Overview

Activity log

Access control (IAM)

Tags

Resource visualizer

Settings

Automation

Help

Essentials

Resource group (move)

Copy to clipboard

Location

East US

Subscription (move)

LabsKraft

Subscription ID

d008c7cc-9795-4292-bf79-74ae52cda63d

Tags (edit)

Add tags

State

Succeeded

Resource ID

/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/...

JSON View

Add or remove favorites by pressing Ctrl+Shift+F

4. Given Role Assignment

Home > keshs2531 | Access Control (IAM) >

Add role assignment

RoleMembersConditionsReview + assign

Role

Storage Blob Data Contributor

Scope

/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/LabsKraft2027/providers/Microsoft.Storage/storageAccounts/keshs2531

Members

Name	Object ID	Type
keshconnector	a5fca583-e331-49c7-a4fb-51b33ee74458	Access Connector for Azure Databricks ⓘ

Description

No description

Condition

None

Review + assignPreviousNext

Feedback

5. Created Synapse Workspace

Home > Microsoft.Azure.SynapseAnalytics-20250825104737 | Overview ...

Deployment

Search x << Delete Cancel Redeploy Download Refresh

Overview

Inputs

Outputs

Template

✓ Your deployment is complete

Deployment name : Microsoft.Azure.SynapseAnaly... Start time : 8/25/2025, 10:49:13 AM
Subscription : LabsKraft Correlation ID : efe3a372-fc46-424d-aeb0-b3e...
Resource group : LabsKraft2027

> Deployment details

Next steps

[Go to resource group](#)

Give feedback

[Tell us about your experience with deployment](#)

Cost management
Get notified to stay within your budget and prevent unexpected charges on your bill.
[Set up cost alerts >](#)

Microsoft Defender for Cloud
Secure your apps and infrastructure
[Go to Microsoft Defender for Cloud >](#)

Free Microsoft tutorials
[Start learning today >](#)

Work with an expert
Azure experts are service provider partners who can help manage your assets on Azure and be your first line of support.
[Find an Azure expert >](#)

Add or remove favorites by pressing Ctrl+Shift+F

Home > Microsoft.Azure.SynapseAnalytics-20250825104737 | Overview > LabsKraft2027 >

keshneww Synapse workspace

Search << + New dedicated SQL pool + New Apache Spark pool Refresh Reset SQL admin password Delete

Suggest a workload for this Synapse workspace

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Resource visualizer

Settings

Analytics pools

Security

Monitoring

Automation

Help

Essentials

Resource group ([move](#))
[LabsKraft2027](#)

Status
Succeeded

Location
France Central

Subscription ([move](#))
[LabsKraft](#)

Subscription ID
d008c7cc-9795-4292-bf79-74ae52cda63d

Managed virtual network
No

Managed Identity object ID
d80a64a4-7617-4db8-951e-0cf13c38b079

Workspace web URL
<https://web.azuresynapse.net?workspace=%2fsubscriptions%2fd008c7cc-...>

Tags ([edit](#))
[Add tags](#)

Networking
[Show firewall settings](#)

Primary ADLS Gen2 account URL
<https://keshs2531.dfs.core.windows.net>

Primary ADLS Gen2 file system
raw

SQL admin username
sqladminuser

SQL Microsoft Entra admin
tredence10@michaelkloudkraft@hotmail.onmicrosoft.com

Dedicated SQL endpoint
keshneww.sql.azuresynapse.net

Serverless SQL endpoint
keshneww-ondemand.sql.azuresynapse.net

Development endpoint
<https://keshneww.dev.azuresynapse.net>

JSON View

Getting started

6. Created Event Hub

The screenshot shows the Azure portal interface for the 'keshevent' Event Hubs Namespace. The left sidebar contains navigation links: Overview, Activity log, Access control (IAM), Tags, Diagnose and solve problems, Data Explorer, Resource visualizer, Events, Settings, Entities, Monitoring, Automation, and Help. The main content area is titled 'Overview' and includes a search bar, a list of actions (+ Event Hub, Delete, Refresh, Give feedback), and a 'JSON View' link. The 'Essentials' section displays the following details:

- Resource group: [\(move\) LabsKraft2027](#)
- Status: Succeeded
- Location: France Central
- Subscription: [\(move\) LabsKraft](#)
- Subscription ID: d008c7cc-9795-4292-bf79-74ae52cda63d
- Host name: keshevent.servicebus.windows.net
- Tags: [\(edit\)](#) [Add tags](#)
- Created: Monday, August 25, 2025 at 11:00:29 GMT+5:30
- Updated: Monday, August 25, 2025 at 11:00:51 GMT+5:30
- Zone Redundancy: Enabled
- Pricing tier: [Basic](#)
- Throughput Units: [1 unit](#)
- Auto-inflate throughput units: [Not Supported](#)
- Local Authentication: [Enabled](#)

Below the Essentials section, there are three status indicators:

- NAMESPACE CONTENTS: 0 EVENT HUBS
- KAFKA SURFACE: NOT SUPPORTED
- ZONE REDUNDANCY: ENABLED

At the bottom, there is a 'Show data for the last:' filter with options: 1 hour, 6 hours, 12 hours, 1 day, 7 days, and 30 days.

7. Created a new catalog's External location

The screenshot shows the Databricks interface with a 'Test connection' dialog box open. The background shows the 'Catalog Explorer' with 'keshcatalog' selected. The dialog box has a title bar 'Test connection' and a close button. It contains the following information:

- Location Type: Directory
- Success - Read
- Success - List
- Success - Write
- Success - Delete
- Success - Path Exists
- Success - Hierarchical Namespace Enabled
- All Permissions Confirmed: The associated Storage Credential grants permission to perform all necessary operations.
- Done button

The background interface includes a sidebar with navigation links: New, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie, Alerts, Query History, SQL Warehouses, Data Engineering, Job Runs, Data Ingestion, and AI/ML. The main content area shows the 'Catalog Explorer' with 'keshcatalog' selected, and a 'Test connection' button.

8. Created catalog

The screenshot shows the Databricks Catalog Explorer interface. On the left, the 'Catalog' sidebar is open, showing a tree view of catalogs under 'My organization'. The 'keshcat' catalog is selected. The main panel displays the 'keshcat' catalog details, including a search bar for schemas, a table of schemas, and sections for 'About this catalog' and 'Tags'.

Catalog Explorer > keshcat

Overview Details Permissions Workspaces

Description

AI generate Add

Filter schemas 2 schemas

Name	Owner	Created at
default	tredence10@michaelkloudkraft...	Aug 25, 2025, 11:23 AM
information_schema	System user	Aug 25, 2025, 11:23 AM

About this catalog

Owner Tredence LabsKraft10

Tags

Add tags

9. Created bronze, silver and gold schema

The screenshot shows the Databricks Catalog Explorer interface. On the left, the 'Catalog' sidebar is open, showing a tree view of catalogs under 'My organization'. The 'capstone_retail_kesh' schema is selected. The main panel displays the 'capstone_retail_kesh' schema details, including a search bar for tables, a table of tables, and sections for 'About this schema' and 'Tags'.

Catalog Explorer > keshcat > capstone_retail_kesh

Overview Details Permissions

Description

AI generate Add

Filter tables Tables 3 Volumes 0 Models 0 Functions 0 Sort

Name	Owner	Created at	Popularity
bronze_orders	tredence10@michaelkl...	Aug 25, 2025, 12:38 PM	----
gold_sales_summary	tredence10@michaelkl...	Aug 25, 2025, 12:42 PM	----
silver_orders	tredence10@michaelkl...	Aug 25, 2025, 12:42 PM	----

About this schema

Owner Tredence LabsKraft10

Tags

Add tags

10. Created bronze , silver and gold layer pipeline

```
from dlt import table
import pyspark.sql.functions as F
from pyspark.sql.types import StructType, StructField, StringType, DoubleType,
IntegerType, TimestampType
# :white_check_mark: Define schema for retail IoT CSV
iot_schema = StructType([
    StructField("OrderID", StringType(), True),
    StructField("CustomerID", StringType(), True),
    StructField("Region", StringType(), True),
    StructField("StoreID", StringType(), True),
    StructField("Product", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", DoubleType(), True),
    StructField("SalesAmount", DoubleType(), True),
    StructField("OrderTime", StringType(), True),      # can cast to Timestamp
    # later in Silver
    StructField("DeviceType", StringType(), True),
    StructField("Temperature", DoubleType(), True),
    StructField("DeviceStatus", StringType(), True),
    StructField("AnomalyFlag", IntegerType(), True),
    StructField("AnomalyType", StringType(), True)
])
# :white_check_mark: Paths (use subdirectories to avoid overlap with DLT system
# files)
raw_path = "abfss://raw@keshs2531.dfs.core.windows.net/raw_data/"
schema_path =
"abfss://raw@keshs2531.dfs.core.windows.net/dlt_schemas/bronze_orders_hetul"
checkpoint_path =
"abfss://raw@keshs2531.dfs.core.windows.net/dlt_checkpoints/bronze_orders_hetul"
"
@table(
    name="keshcat.capstone_retail_kesh.bronze_orders",
    comment="Bronze table - raw ingested IoT retail data via Auto Loader"
)
def bronze_orders():
```

```

    return (
        spark.readStream
            .format("cloudFiles")
            .option("cloudFiles.format", "csv")
            .option("header", True)
            .option("cloudFiles.schemaLocation", schema_path)
            .schema(iot_schema)    # :white_check_mark: manually set schema (no
infer needed)
            .load(raw_path)        # :white_check_mark: points to /raw_data/
folder
    )

```

```

from dlt import table
import pyspark.sql.functions as F
from pyspark.sql.types import DoubleType, IntegerType
# =====
# Silver Layer - Cleaned IoT Orders
# =====
@table(
    name="capstone_retail_kesh.silver_orders",
    comment="Silver layer: cleaned and deduplicated IoT orders with anomaly
handling"
)
def silver_orders():
    # Read Bronze
    orders = dlt.read("bronze_orders")
    # Standardize schema
    orders = (
        orders.withColumn("Quantity", F.col("Quantity").cast(IntegerType()))
            .withColumn("UnitPrice", F.col("UnitPrice").cast(DoubleType()))
            .withColumn("SalesAmount",
F.col("SalesAmount").cast(DoubleType()))
            .withColumn("Temperature",
F.col("Temperature").cast(DoubleType()))
            .withColumn("OrderTime", F.to_timestamp("OrderTime", "M/d/yyyy
H:mm"))
    )

```

```

# Deduplicate by OrderID
orders = orders.dropDuplicates(["OrderID"])
# Handle missing anomaly type
orders = orders.withColumn(
    "AnomalyType",
    F.when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType").isNull()),
"Unknown")
        .when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType") == ""),
"Unknown")
        .otherwise(F.col("AnomalyType"))
    )
# Add business rule fields
orders = (
    orders.withColumn("IsAnomaly", F.col("AnomalyFlag") == 1)
        .withColumn("DeviceStatus", F.upper(F.col("DeviceStatus")))
        .withColumn("NormalizedStoreID", F.upper(F.col("StoreID")))
        .withColumn("NormalizedProduct", F.upper(F.col("Product")))
    )
return orders

```

```

from dlt import table, read
import pyspark.sql.functions as F
# =====
# Gold Layer Tables (capstone_retail.gold)
# =====
# 1. Daily / Monthly Sales per Region
@table(
    name="keshcat.capstone_retail_kesh.gold_sales_region",
    comment="Daily & Monthly Sales per Region"
)
def gold_sales_region():
    return (
        read("keshcat.capstone_retail_kesh.silver_orders")
        .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
        .withColumn("YearMonth", F.date_format("OrderDate", "yyyy-MM"))
        .groupBy("Region", "OrderDate", "YearMonth")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )

```



```

    )
# 2. Device Anomaly Trend per Store
@table(
    name="keshcat.capstone_retail_kesh.gold_anomaly_trends",
    comment="Device Anomaly Trend per Store"
)
def gold_anomaly_trends():
    return (
        read("keshcat.capstone_retail_kesh.silver_orders")
        .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
        .groupBy("StoreID", "OrderDate", "DeviceType", "AnomalyType")
        .agg(F.count("*").alias("AnomalyCount"))
    )
# 3. Conversion Rate Impact when devices fail
@table(
    name="keshcat.capstone_retail_kesh.gold_conversion_impact",
    comment="Conversion rate impact of device failures"
)
def gold_conversion_impact():
    df = read("keshcat.capstone_retail_kesh.silver_orders")
    total_orders = df.groupBy("StoreID").agg(F.count("*").alias("TotalOrders"))
    failed_orders = (
        df.filter(df.AnomalyFlag == 1)
        .groupBy("StoreID")
        .agg(F.count("*").alias("FailedOrders"))
    )
    return (
        total_orders.join(failed_orders, "StoreID", "left")
        .withColumn("FailedOrders", F.coalesce(F.col("FailedOrders"),
F.lit(0)))
        .withColumn("FailureRate", F.col("FailedOrders") /
F.col("TotalOrders"))
    )
# 4. Store Tier Classification
@table(
    name="keshcat.capstone_retail_kesh.gold_store_tiers",
    comment="Store tiers based on total sales (High / Medium / Low performers)"
)

```

```

def gold_store_tiers():
    df = (
        read("keshcat.capstone_retail_kesh.silver_orders")
        .groupBy("StoreID")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )
    return (
        df.withColumn(
            "StoreTier",
            F.when(F.col("TotalSales") > 100000, "High")
            .when(F.col("TotalSales") > 50000, "Medium")
            .otherwise("Low")
        )
    )

# 5. Weighted Average Sales vs Anomalies
@table(
    name="keshcat.capstone_retail_kesh.gold_weighted_sales_anomalies",
    comment="Weighted average of sales vs anomalies per store"
)

def gold_weighted_sales_anomalies():
    df = read("keshcat.capstone_retail_kesh.silver_orders")
    return (
        df.groupBy("StoreID")
        .agg(
            F.sum("SalesAmount").alias("TotalSales"),
            F.sum(F.when(F.col("AnomalyFlag") == 1,
1).otherwise(0)).alias("TotalAnomalies")
        )
        .withColumn("SalesPerAnomaly",
            F.when(F.col("TotalAnomalies") > 0, F.col("TotalSales") /
F.col("TotalAnomalies"))
            .otherwise(F.lit(None))
        )
    )

```

Microsoft Azure databricks

Search data, notebooks, recents, and more... CTRL + P

tred_d3

+ New

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie

Alerts

Query History

SQL Warehouses

Data Engineering

Job Runs

Data Ingestion

AI/ML

Jobs & Pipelines >

keshps ☆ Send feedback

Development Production Settings Schedule Share Start

Run: 25/8/2025, 1:02:47 pm · Completed Select tables for refresh

Graph List

Pipeline details Update details

Pipeline ID 0e6c70c5-4b10-4c7a-8a4b-fd6888949eb2

Pipeline type ETL pipeline

Source code /Users/tredence10@michaelkloudkraft@hotmail.onmicrosoft.com/kesh_cap

Event log Query history

All Info Warning Error Filter...

2 minutes ago user_action User tredence10@michaelkloudkraft@hotmail.onmicrosoft.com started an update.

2 minutes ago create_update Update 320c22 started by USER_ACTION.

2 minutes ago update_progress Update 320c22 is INITIALIZING.

2 minutes ago update_progress Update 320c22 Show new events

NIFTY +0.45%

Search

ENG IN 13:05 25-08-2025

12.PowerBi visualisation

Untitled - Power BI Desktop KESH SINGH

File Home Insert Modeling View Optimize Help

Paste Copy Cut Format painter

Get data Excel OneLake SQL Server Enter data Datawarehouse Recent sources Transform Refresh

New visual Text box More visuals

New visual calculation New measure Quick measure

Sensitivity Publish

Prep data for Copilot AI

Clipboard

Sum of AnomalyCount by AnomalyType and DeviceType

DeviceType POS_Terminal RFID_Scanner Temp_Sensor

Sum of AvgQuantity, Sum of TotalSales and Sum of UniqueCustomers by Product and Region

Region East North South West

Sum of SalesPerAnomaly, Sum of TotalAnomalies and Sum of TotalSales by StoreID

StoreID StoreTier Sum of TotalSales

Visualizations

Build visual

Filters

Values

Add data fields here

Drill through

Cross-report

Keep all filters

Add drill-through fields here

Page 1

Page 1 of 1

13. Virtual machine created

Home > CreateVm-canonical.ubuntu-24.04-lts-server-20250825201240 | Overview >

keshvm Virtual machine

Help me copy this VM in any region Manage this VM with Azure CLI

Search

Connect Start Restart Stop Hibernate Capture Delete Refresh Open in mobile Feedback

Overview

- Activity log
- Access control (IAM)
- Tags
- Diagnose and solve problems
- Resource visualizer
- Connect
- Networking
- Settings
- Availability + scale
- Security
- Backup + disaster recovery
- Operations
- Monitoring
- Automation

Essentials

Resource group (move) [LabsKraft2027](#)

Status: Running

Location: France Central

Subscription (move) [LabsKraft](#)

Subscription ID: d008c7cc-9795-4292-bf79-74ae52cda63d

Tags (edit) [Add tags](#)

Operating system: Linux (ubuntu 24.04)

Size: Standard B1s (1 vcpu, 1 GiB memory)

Public IP address: [4.233.70.92](#)

Virtual network/subnet: [keshvm-vnet/default](#)

DNS name: [Not configured](#)

Health state: -

Time created: 8/25/2025, 2:47 PM UTC

[JSON View](#)

Properties Monitoring Capabilities (7) Recommendations Tutorials

<https://portal.azure.com/#home> `Ctrl+Shift+F`

14. Created event hub and shared access policy

Microsoft Azure Search resources, services, and docs (G+/) Copilot tredence10@michaelklo... DEFAULT DIRECTORY (MICHAELKLO...)

Home > keshevent

keshevent | Shared access policies

Event Hubs Namespace

Search

+ Add

Search to filter items...

Policy

- [RootManageSharedAccessKey](#)
- [manage](#)

SAS Policy: manage

Event Hub

Regenerate primary key Regenerate secondary key Delete

Event Hubs access control with Shared Access Signatures [Learn more](#)

Claims

- ☒ Manage
- ☒ Send
- ☒ Listen

Primary key

.....

Secondary key

.....

Primary connection string

.....

Secondary connection string

.....

SAS policy ARM ID

/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/abcKr

Save Discard changes Give feedback

<https://portal.azure.com/#> `Ctrl+Shift+F`


```

    StructField("Product", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", DoubleType(), True),
    StructField("SalesAmount", DoubleType(), True),
    StructField("OrderTime", StringType(), True),
    StructField("DeviceType", StringType(), True),
    StructField("Temperature", DoubleType(), True),
    StructField("DeviceStatus", StringType(), True),
    StructField("AnomalyFlag", IntegerType(), True),
    StructField("AnomalyType", StringType(), True)
])

raw_path = "abfss://raw@keshs2531.dfs.core.windows.net/incoming/"
schema_path =
"abfss://raw@keshs2531.dfs.core.windows.net/dlt_schemas/bronze_orders_kesh"

@table(
    name="keshcat.bronze.bronze_orders_csv",
    comment="Bronze table - raw IoT CSV"
)

def bronze_orders_csv():
    return (
        spark.readStream
            .format("cloudFiles")
            .option("cloudFiles.format", "csv")
            .option("header", True)
            .option("cloudFiles.schemaLocation", schema_path)
            .schema(iot_schema)
            .load(raw_path)
            .withColumn("OrderTime", F.to_timestamp("OrderTime"))
    )

# =====
# ② Bronze Layer: Event Hub IoT
# =====
eventhub_connection_string =
"Endpoint=sb://keshevent.servicebus.windows.net/;SharedAccessKeyName=manage;Share
dAccessKey=5T5qX8qfWXpC3l734xBKItCaQTZ0AN23v+AEhBDM89w=;EntityPath=keshehub"

```

```

consumer_group = "$Default"

ehConf = {
    'eventhubs.connectionString': eventhub_connection_string,
    'eventhubs.consumerGroup': consumer_group,
    'eventhubs.startingPosition': '@latest'
}

@table(
    name="keshcat.bronze.bronze_orders_eventhub",
    comment="Bronze table - live IoT data from Event Hub"
)

def bronze_orders_eventhub():
    df = spark.readStream \
        .format("eventhubs") \
        .options(**ehConf) \
        .load()

    # Event Hub body is bytes → convert to string → parse JSON
    df1 = df.selectExpr("cast(body as string) as json_string")
    df_parsed = df1.select(F.from_json(F.col("json_string"),
iot_schema).alias("data")).select("data.*")
    return df_parsed.withColumn("OrderTime", F.to_timestamp("OrderTime"))

# =====
# ③ Unified Bronze Table
# =====

@table(
    name="keshcat.capstone_retail_kesh.bronze_orders",
    comment="Unified Bronze table (CSV + Event Hub)"
)

def bronze_orders():
    df_csv = read("keshcat.capstone_retail_kesh.bronze_orders_csv")
    df_eh = read("keshcat.capstone_retail_kesh.bronze_orders_eventhub")
    return df_csv.unionByName(df_eh)

# =====
# ④ Silver Layer
# =====

```

```

@table(
    name="keshcat.capstone_retail_kesh.silver_orders",
    comment="Silver layer: cleaned and deduplicated IoT orders with anomaly
handling"
)

def silver_orders():
    orders = read("keshcat.capstone_retail_kesh.bronze_orders")

    orders = (
        orders.withColumn("Quantity", F.col("Quantity").cast(IntegerType()))
            .withColumn("UnitPrice", F.col("UnitPrice").cast(DoubleType()))
            .withColumn("SalesAmount",
F.col("SalesAmount").cast(DoubleType()))
            .withColumn("Temperature",
F.col("Temperature").cast(DoubleType()))
            .withColumn("OrderTime", F.to_timestamp("OrderTime", "M/d/yyyy
H:mm"))
    )

    orders = orders.dropDuplicates(["OrderID"])

    orders = orders.withColumn(
        "AnomalyType",
        F.when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType").isNull()),
"Unknown")
            .when((F.col("AnomalyFlag") == 1) & (F.col("AnomalyType") == ""),
"Unknown")
            .otherwise(F.col("AnomalyType"))
    )

    orders = (
        orders.withColumn("IsAnomaly", F.col("AnomalyFlag") == 1)
            .withColumn("DeviceStatus", F.upper(F.col("DeviceStatus")))
            .withColumn("NormalizedStoreID", F.upper(F.col("StoreID")))
            .withColumn("NormalizedProduct", F.upper(F.col("Product")))
    )

    return orders

```



```

# =====
# ⑤ Gold Layer
# =====

@table(
    name="keshcat.capstone_retail_kesh.gold_sales_region",
    comment="Daily & Monthly Sales per Region"
)

def gold_sales_region():
    return (
        read("keshcat.capstone_retail_kesh.silver_orders")
        .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
        .withColumn("YearMonth", F.date_format("OrderDate", "yyyy-MM"))
        .groupBy("Region", "OrderDate", "YearMonth")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )

@table(
    name="keshcat.capstone_retail_kesh.gold_anomaly_trends",
    comment="Device Anomaly Trend per Store"
)

def gold_anomaly_trends():
    return (
        read("keshcat.capstone_retail_kesh.silver_orders")
        .withColumn("OrderDate", F.to_date("OrderTime", "M/d/yyyy H:mm"))
        .groupBy("StoreID", "OrderDate", "DeviceType", "AnomalyType")
        .agg(F.count("*").alias("AnomalyCount"))
    )

@table(
    name="keshcat.capstone_retail_kesh.gold_conversion_impact",
    comment="Conversion rate impact of device failures"
)

def gold_conversion_impact():
    df = read("keshcat.capstone_retail_kesh.silver_orders")
    total_orders = df.groupBy("StoreID").agg(F.count("*").alias("TotalOrders"))
    failed_orders = (
        df.filter(df.AnomalyFlag == 1)

```

```

        .groupBy("StoreID")
        .agg(F.count("*").alias("FailedOrders"))
    )
    return (
        total_orders.join(failed_orders, "StoreID", "left")
        .withColumn("FailedOrders", F.coalesce(F.col("FailedOrders"),
F.lit(0)))
        .withColumn("FailureRate", F.col("FailedOrders") /
F.col("TotalOrders"))
    )

@table(
    name="keshcat.capstone_retail_kesh.gold_store_tiers",
    comment="Store tiers based on total sales (High / Medium / Low performers)"
)
def gold_store_tiers():
    df = (
        read("keshcat.capstone_retail_kesh.silver_orders")
        .groupBy("StoreID")
        .agg(F.sum("SalesAmount").alias("TotalSales"))
    )
    return (
        df.withColumn(
            "StoreTier",
            F.when(F.col("TotalSales") > 100000, "High")
            .when(F.col("TotalSales") > 50000, "Medium")
            .otherwise("Low")
        )
    )

@table(
    name="keshcat.capstone_retail_kesh.gold_weighted_sales_anomalies",
    comment="Weighted average of sales vs anomalies per store"
)
def gold_weighted_sales_anomalies():
    df = read("keshcat.capstone_retail_kesh.silver_orders")
    return (
        df.groupBy("StoreID")

```

```
.agg(  
    F.sum("SalesAmount").alias("TotalSales"),  
    F.sum(F.when(F.col("AnomalyFlag") == 1,  
1).otherwise(0)).alias("TotalAnomalies")  
)  
    .withColumn("SalesPerAnomaly",  
        F.when(F.col("TotalAnomalies") > 0, F.col("TotalSales") /  
F.col("TotalAnomalies"))  
        .otherwise(F.lit(None))  
    )  
)
```